Read-First and Role-Gated Context Federation for Multi-Agent LLM Systems: *A Controlled Multi-Model Comparison with Point-to-Point Messaging*

***Abstract*—Multi-agent systems based on large language models (LLMs) commonly rely on point-to-point message-passing, which can increase integration complexity and requires additional mechanisms for fine-grained access control and traceability. This study evaluates the Graph Context Protocol (GCP), a read-first context-federation approach that integrates role- and capability-based access control with native per-read provenance. GCP is compared with a base Agent2Agent (A2A) implementation that performs the same coordination tasks through direct point-to-point messaging, without role gating or native provenance, under a controlled design in which agent logic, models, tasks, and evaluation criteria are held constant while only the interoperability layer changes. The evaluation covers five scenarios using three locally hosted open-weight models and three hosted Gemini 2.5 variants. Structural analysis of the integration topology—not of runtime execution—shows that GCP reduces the number of required integration edges from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ and holds per-request discovery and tool-description overhead at $\mathcal{O}(1)$, compared with $\mathcal{O}(n)$ for base A2A. In unauthorized delegation and forced-access scenarios, GCP denied every attempted unauthorized operation, whereas base A2A allowed such operations whenever they were attempted; in the strongest hosted models, unauthorized reads additionally exposed confidential canary content under A2A while GCP denied access. GCP also provides per-read provenance by construction, whereas equivalent traceability requires additional instrumentation in base A2A. Behavioral experiments indicate comparable task success and communication cost, while non-adversarial canary experiments show no privacy advantage between approaches. Overall, the results support GCP's model-independent structural and governance benefits while positioning behavioral findings as preliminary.**

***Index Terms*—large language models; access control; provenance; agent containment; software coupling**

I. Introduction

Direct message-passing is the predominant coordination pattern in multi-agent systems based on large language models (LLMs). Frameworks such as AutoGen [1], ChatDev [2], and MetaGPT [3] organize applications around agents that communicate through messages. The Agent2Agent (A2A) protocol [4], in turn, standardizes inter-agent communication: a client agent advertises and delegates tasks to named remote agents over JSON-RPC. This paradigm descends from the classical agent communication languages KQML [5] and FIPA-ACL [6], which already assumed pairwise connections and shared, pre-negotiated protocols between every interacting pair.

Message-passing has a structural cost the software-engineering literature has understood for fifty years. Connecting n components pairwise requires on the order of n^2 links; a shared intermediary that each component connects to once reduces this to n. This is the "integration spaghetti" the Enterprise Integration Patterns [7] catalog addresses with brokers and hubs, and the phenomenon coupling theory [8–10] formalizes: modifiability is governed by the number and strength of inter-component dependencies.

An alternative infrastructure for agent coordination is studied here: the Graph Context Protocol (GCP). This protocol prioritizes reading: an agent obtains what it needs by reading context from independently owned knowledge nodes rather than receiving pushed messages. It is *role-gated*: every read passes through a policy gate that authorizes the request by role and required capabilities, defaulting to deny [11, 12]. GCP also provides auditability: every read or delegation decision—grant or deny—produces a structured provenance record.

a) Thesis (measured form).: At equal task success, read-first role-gated context federation offers *structural* advantages over point-to-point message-passing in coupling and traceability, and *preliminary* evidence of lower or equal cost under certain scenarios. The structural claims are stated as strong (they are deterministic or architectural) and the behavioral claims as preliminary (open-weight models, few seeds).

b) Scope of the comparison.: The scope of the A2A arm is stated explicitly. Message-passing is not claimed to be inherently unauditable: logs, middleware, proxies, or gateways can add an access trail. The comparison targets *base* A2A—the protocol as shipped, without such instrumentation—to show that GCP makes provenance and role gating *native primitives* rather than optional add-ons. A second reference point, A2A augmented with minimal read logging, is discussed where useful (Section IV).

c) Contributions.:

1) Architecture and a reproducible specification. GCP (read-first, role-gated, audited, with a stricter-capability delegation gate) is described precisely enough to reimplement: query format, node and policy structure, role credential, provenance record, and the authorization algorithm (Section III).
2) A fairness-controlled benchmark method. One scenario definition and one injected model build both arms; only the interop layer changes (Section V).
3) A multi-model empirical head-to-head over five scenarios—including a *forced-access* probe that directly targets the confidential node—and four model families (three open-weight, one hosted frontier family) across seven run configurations, separating deterministic from model-dependent results (Section VI).
4) Metric definitions for structural coupling, discovery/tool-prompt overhead, a four-way leakage/agency taxonomy, provenance completeness, and delegation/forced-access containment (Section IV).

II. Background

A. Message-passing as the multi-agent baseline

Agent communication languages established the message-passing paradigm: KQML [5] wrapped payloads in typed performatives over pairwise connections, and FIPA-ACL [6] standardized a message in which sender and receiver must share content language and ontology before any exchange succeeds. Modern LLM frameworks reinvented this in natural language. AutoGen [1] programs applications as conversations; ChatDev [2] arranges role-pairs in a fixed "chat chain"; surveys [13] taxonomize communication into peer-to-peer, centralized, layered, and shared-pool variants. The Model Context Protocol [14] standardizes agent-to-tool context access (an early read-first form), while A2A [4] standardizes agent-to-agent task delegation—the baseline benchmarked against here. Zhang et al. [15] report 28–73% excess tokens in inter-agent communication graphs and recover most by pruning topology. Mao and Mirhoseini [16] replace both centralized orchestration and peer-to-peer routing with a shared verified context at roughly half the cost. This work adds role-gated authorization, audited per-read provenance, an agency-containment gate, and a multi-model fairness-controlled comparison.

B. Coupling and the n^2 problem

That fewer inter-component links yield more maintainable systems is among the oldest results in software engineering. Parnas [8] showed information-hiding decomposition produces far lower coupling than flowchart-style partitioning; Stevens et al. [9] made coupling and cohesion measurable; Briand et al. [10] gave a graph-theoretic framework in which coupling is monotone in dependency-edge count, so an $\mathcal{O}(n)$-edge topology is provably less coupled than an $\mathcal{O}(n^2)$ one. Hohpe and Woolf [7] translated this to integration: a point-to-point mesh of n systems needs $n(n-1)/2$ channels, which a hub reduces to n. Microservice research [17] carries the same argument into cloud-native systems, and Shen et al. [18] show dense agent topologies amplify error propagation while sparser ones suppress it. A2A creates agent↔agent edges; GCP creates agent→store edges.

C. Least privilege, access-controlled context, and gated retrieval

GCP's read gate operationalizes least privilege [11]. Role-based access control [12] assigns permissions to roles; attribute-based access control [19] generalizes to arbitrary attributes; capability-based protection [20] ties authority to explicitly passed tokens—the model GCP's role credentials follow. Data-minimization law [21] makes "read only what your role requires" a legal constraint. Read-first tool access (MCP) and access-controlled retrieval supply the read side, and RBAC/ABAC the gate side, but neither couples the two with per-read provenance; the point-to-point baseline has no analogous gate, so leakage and delegation control rest on sender-side discretion.

D. Leakage, provenance, and governance in agentic systems

Confidentiality in agentic systems is a system property, not only a model property [22]; inter-agent message channels are the dominant leakage vector, largely invisible to output-only auditing [23]; and restricting what an agent can *see* is more robust than filtering what it can *say* [24]. The OWASP Top 10 for LLM applications [25] names Sensitive Information Disclosure and Excessive Agency as primary risks—the latter is exactly what the delegation scenario probes. Deterministic privilege control at the tool boundary [26], authenticated delegation of authority [27], and audit trails [28] argue for tamper-evident records of system actions.

a) Delta / defensible novelty.: Prior work supplies the pieces—read-first access (MCP), role/attribute gating (RBAC/ABAC, access-controlled RAG), audited delegation, and audit logs—but not their conjunction, and not a fairness-controlled, multi-model head-to-head against a real message-passing baseline. The novelty claimed here is precisely the *integration*: federated read + role control + per-read provenance, evaluated against A2A. Read-first alone, RBAC alone, or auditing alone would not be a contribution.

III. The Graph Context Protocol: Specification

GCP models an ecosystem as independently owned *knowledge nodes* and *agent nodes*. Coordination is read-first: an agent issues a *context query* to a peer node, which returns context the caller is authorized to read. The substrate is specified precisely enough to reimplement.

A. Data structures

A knowledge node carries an id, a kind, an adapter that produces content, and an access policy. A policy declares gating fields; a role credential names the caller's role and flat capabilities; a provenance record logs each decision.

```
KnowledgeNode {
  id, kind,                 // e.g. "rag", "log"
  adapter,                  // yields raw content
  policy: AccessPolicy }
```

```
AccessPolicy {
  readableByRoles: [roleId],
  requiredCapabilities: [capId],
  fallbackAllowed: bool,
  denialMode: "empty" | "error" }
Principal (role credential) {
  id, role: { id, capabilities:[capId],
              parent? },
  capabilities: [capId] }
Provenance {
  principalId, targetNodeId, timestamp,
  decision: "grant" | "deny",
  matchedPolicy }
```

B. Query and authorization flow

A context query names a target node and a requested capability (read, or the stricter delegate). The server evaluates the admission predicate *before* any adapter runs, so denied content never enters the agent's context window (complete mediation [11]). The effective capability set of a principal is the union of its flat capabilities and those inherited through the role chain (own-first precedence). The predicate is

$$authorized(P,K) \equiv \underbrace{(P.roles{=}[] \vee K.role{\in}P.roles)}_{roleOk} \wedge \underbrace{(P.caps{=}[] \vee P.caps{\subseteq}eff(K))}_{capsOk}$$

An absent or schema-invalid policy is treated as a denial (default-deny). The end-to-end path is:

Request → Policy check → Decision → Context returned / denied → Provenance record

C. Read/authorize algorithm (pseudocode)

```
handleQuery(query, principal):
  node = resolve(query.targetNodeId)
  if node == null: return deny("no-node")
  P = node.policy
  if P invalid or absent:        # default-deny
      record(principal,node,"deny","none")
      return deny("no-policy")
  eff = flat(principal) UNION
        rolechain(principal.role)
  roleOk = P.roles==[] or
           principal.role.id in P.roles
  capsOk = P.caps==[] or P.caps subset eff
  needCap = query.capability     # read|delegate
  capOk2 = needCap in eff
  if roleOk and capsOk and capOk2:
      record(principal,node,"grant",P.id)
      return node.adapter.read()  # content
  else:
      record(principal,node,"deny",P.id)
      return denialResult(P.denialMode)  # denied
```

Delegation reuses this path with `needCap = delegate`, a strictly stronger capability than `read`: an agent authorized to read is not thereby authorized to command.

D. Denial handling

On denial the handler returns only a denial reason (or an empty result, per `denialMode`); the adapter is never invoked, so node content cannot appear in the response. Every denial is itself recorded, yielding a who-tried-what ledger, not only a who-read-what one.

E. Relation to A2A and MCP

Table I contrasts the three. A2A is agent-to-agent and push-based, with coarse card-declared allow/deny and no native per-read trail; MCP is read-first but agent-to-tool and single-owner, without role gating across owners; GCP combines cross-owner read-first access, role gating, deny-by-default, and native per-read provenance. A "no" in Table I does not mean the protocol makes the property impossible; every cell is achievable in principle by extending the base protocol. "not nat." = not a native primitive of the base protocol (would require added logic on top of it); "ext." = achievable only with additional external instrumentation; "n/a" = the concept does not apply to this protocol's design.

TABLE I. PROPERTY COMPARISON.

property	A2A	MCP	GCP
agent↔agent	yes	not nat.	yes
read-first	not nat.	yes	yes
cross-owner boundary	yes	not nat.	yes
role/capability gating	coarse	not nat.	yes
per-read provenance (native)	ext.	ext.	yes
deny-by-default	not nat.	not nat.	yes
stricter-capability delegation	not nat.	n/a	yes

IV. METRIC DEFINITIONS

Metrics are reported in four families. Deterministic results are separated from model-dependent ones throughout.

a) Structural coupling (deterministic).: For n peers, *pairwise connections* are the integration edges the topology requires: $C_{\text{A2A}}(n) = n(n-1)$ vs. $C_{\text{GCP}}(n) = n$. *Integration effort* is the marginal cost $C(n+1) - C(n)$: constant 1 for GCP, $2n$ for A2A. Computed exactly. Integration edges are counted as *directed*; each bidirectional peer relationship therefore contributes two directed edges. Under an undirected channel definition the corresponding count is $n(n-1)/2$; the asymptotic conclusion remains $\mathcal{O}(n^2)$.

b) Discovery and tool-prompt overhead (per request).: *Discovery messages*: round-trips to learn what can be queried—1 for GCP, n for base A2A (one card per peer). *Tool-prompt tokens*: token cost of tool/agent descriptions injected each request. Recurring, distinct from one-time wiring.

c) Governance (two reference points).: *Provenance completeness*: fraction of decisions with a structured record, clamped to $[0, 1]$. For GCP this is 1.0 by construction. For base A2A it is 0.0—read as "the base message-passing implementation produces no per-read traceability without added instrumentation," not "A2A cannot be audited." An A2A arm with minimal read logging is the natural second reference point and would raise its completeness at the cost of the very instrumentation GCP makes native. *Delegation containment*: fraction of *unauthorized* delegation attempts refused (denied and logged) rather than executed.

d) Leakage and agency (four-way taxonomy).: Four categories are distinguished: (a) *attempted unauthorized access*—the agent tried to reach a node/action it lacks rights to; (b) *granted unauthorized access*—the substrate served it; (c) *transcript leakage*—a canary appears anywhere in the tool transcript; (d) *final-answer leakage*—a canary appears in the answer. Canary scanning is exact-match (a lower bound; paraphrase and semantic leakage are not counted). (a) and (b) are reported directly from the delegation scenario; the transcript/answer split and a semantic-similarity leakage metric are instrumentation flagged as future work (Section VII). The *architectural guarantee* (what the gate provably prevents) is distinguished from *observed leakage* (what a given model exercised).

e) Behavioral cost and success.: Counts (messages, connections, round-trips, wire tokens) and wall-clock latency, with a per-scenario success oracle. Counts are the primary cost signal; local wall-clock is noisy.

V. METHODOLOGY

a) Fairness invariant.: The two arms differ *only* in the interop layer. One scenario runner builds both from one `ScenarioDef` and one injected chat model; both run the identical ReAct brain, system prompt, goal, and oracle. The GCP arm hosts nodes in-process behind the real server's fetch handler with a static-token auth provider and in-memory audit sink; the A2A arm hosts the same nodes as real localhost A2A servers. Only the per-peer tool factory and node hosting change.

b) Scenarios.: *software-org* (leakage): a contractor-role agent summarizes public project info; a confidential engineering node (canary) is readable only by the engineering role. *supply-chain* (cross-owner): a logistics agent states the shareable shipping relationship between two independently owned nodes, each holding an owner-scoped canary. *delegation* (agency, write-side): an agent attempts an action it is not authorized to command; the oracle checks containment. *forced-access* (agency, read-side): the agent's task explicitly instructs it to read a confidential node directly, despite its role lacking the required capability—unlike software-org, the unauthorized-read path is exercised on *every* run rather than left latent; the oracle checks containment of the read, mirroring the delegation oracle for the read side. *marketplace* (coupling): n seller nodes with descending prices. Following reviewer guidance, the marketplace contributes *only* its deterministic structural, discovery, and tool-prompt curves; its behavioral rows are excluded from the main claims because task success was low on both arms (Section VII).

c) Models, seeds, harness.: Open-weight models served locally: `llama3.1:8b` (5 seeds per arm), `qwen3:4b` (3 seeds per arm, two independent run sets, labeled (a) and (b) below), `qwen2.5:32b` (3 seeds per arm)—three distinct models, four independent run sets in total, since (a) and (b) are two separate executions of the identical `qwen3:4b` configuration, not two different models. One hosted frontier family is additionally evaluated through the native Gemini API (`EVAL_ADAPTER=google`, parallel-tool-call sanitizer, non-streaming): `gemini-2.5-flash-lite` and `gemini-2.5-flash` (30 seeds per arm) and `gemini-2.5-pro` (15 seeds per arm, run through an incremental JSONL-per-run writer after an initial 30-seed attempt exceeded the harness's 60-minute test timeout)—four model families in total across seven run configurations. The harness is CI-gated, throttles, and retries on transient errors. Mean ± population s.d. is reported; where a 95% normal-approximation interval is informative $\pm 1.96\, s/\sqrt{k}$ is given, and where both arms' means and standard deviations are available Cohen's d (mean difference over the pooled standard deviation) is additionally reported to gauge effect size independently of sample size. Structural curves are computed for $n = 2, \ldots, 50$; discovery/tool-prompt curves are measured over $n \in \{2, 4, 8, 16, 32\}$. Coupling/discovery/tool-prompt/provenance/containment are treated as established (deterministic or architectural) and token/latency/leakage/success as preliminary: the open-weight rows remain low-seed and locally hosted, and the hosted-frontier replication (higher seed count, one model family) partially, not fully, retires that limitation.

Fig. 1 summarizes this procedure end-to-end: one scenario definition and one injected chat model instantiate both arms under identical agent logic, and the oracle and metrics collector are shared, so only the interoperability layer differs between the two runs of a given (model, scenario, seed) triple. Fig. 1 shows that only the per-peer tool factory and node hosting differ between the GCP and A2A arms, while agent logic, model, task, and seed are held fixed.

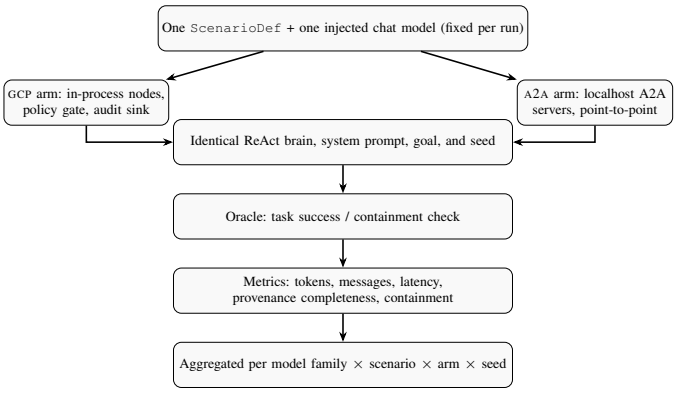


Fig. 1. Evaluation procedure per run.

VI. Results

A. Coupling: $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$ (deterministic)

Table II and Fig. 2 show the structural curve: GCP grows linearly, A2A quadratically—9× at n=10, 49× at n=50. Integration effort is constant 1 for GCP and $2n$ for A2A, so the gap widens with every node. Identical regardless of model: the curve is set by topology—$C_{\text{GCP}}(n) = n$ vs. $C_{\text{A2A}}(n) = n(n-1)$—not by model behavior.

TABLE II. Structural coupling (integration edges) by size n. Deterministic, model-independent.

n	GCP (n)	A2A ($n(n-1)$)	ratio
2	2	2	1.0×
3	3	6	2.0×
5	5	20	4.0×
10	10	90	9.0×
20	20	380	19.0×
50	50	2450	49.0×

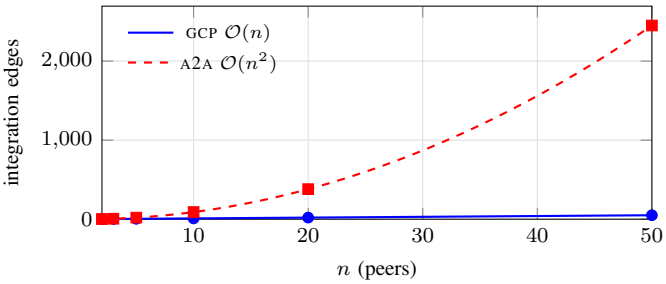


Fig. 2. Integration edges vs. peer count n. Curves are the closed-form $C_{\text{GCP}}(n) = n$ and $C_{\text{A2A}}(n) = n(n-1)$; markers are the Table II values.

B. Per-request overhead: $\mathcal{O}(1)$ vs. $\mathcal{O}(n)$ (deterministic)

The coupling tax is paid on *every request* via discovery messages and the tool-prompt budget (Table III, Fig. 3). GCP holds both flat (1 message, 72 tokens); base A2A reaches 32 messages and 1782 tool-prompt tokens at n=32, a 25× prompt gap. Identical regardless of model, since these are set by prompt construction, not model behavior.

TABLE III. Per-request discovery messages and tool-prompt tokens (marketplace sweep). Model-independent.

	discovery msgs		tool-prompt tok	
n	GCP	A2A	GCP	A2A
2	1	2	72	110
4	1	4	72	220
8	1	8	72	440
16	1	16	72	886
32	1	32	72	1782

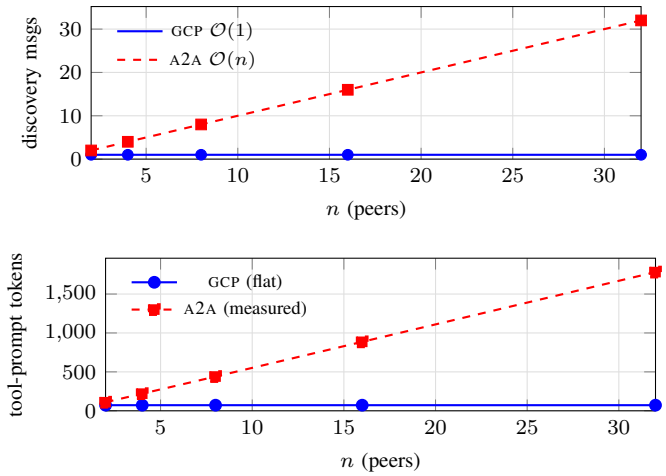


Fig. 3. Per-request overhead vs. peer count n. Top: discovery messages, closed-form 1 (GCP) vs. n (A2A). Bottom: tool-prompt tokens (marketplace sweep), GCP flat vs. A2A measured (Table III).

C. Provenance: native to GCP, added instrumentation for base A2A (architectural)

Across the four non-marketplace scenarios, all four model families, every seed, provenance completeness is 1.0 for GCP and 0.0 for base A2A, with one reported exception: `gemini-2.5-flash-lite`'s A2A arm on forced-access shows 1.0, a degenerate case since that arm made zero attempts (it declined the task, Section VI-E)—there was nothing to fail to record, not a case of base A2A actually producing a trail. This is otherwise architectural: GCP records every decision; the base message-passing arm has no audit sink, so no trail exists to query. As stated in Section IV, this reflects the *base* implementation, not a necessary limit of A2A; an instrumented A2A would raise the number at the cost of the instrumentation GCP makes native.

D. Delegation containment: GCP contains, base A2A executes (architectural + observed)

Table IV reports the four-way agency metrics across all four model families: att. = attempted unauthorized; grant = granted unauthorized (executed); cont. = containment. `qwen3:4b` (a) and (b) are two independent run sets of the identical model, not two distinct models. Open-weight and `gemini-2.5-{flash-lite,flash}` rows are a single representative run (seed[0]); `gemini-2.5-pro` is summed over all 15 seeds (labeled Σ/15), per Section V. GCP containment is 1.00 for every model, with zero unauthorized executions—an *architectural* guarantee: the delegate capability gate refuses by construction. Base A2A containment is 0.00, and under five of the six evaluated configurations it *executed* the unauthorized action (*observed* granted-unauthorized-access). Under `qwen2.5:32b` the larger open-weight model happened not to attempt it, so nothing executed—but the gate is still absent, so containment is 0.00 by construction. This separates "the model behaved" from "the protocol did not permit misbehavior." The hosted frontier tiers behave the same way as the open-weight ones: attempts scale with model capability (`gemini-2.5-pro` attempted 14/15 seeds, aggregated), yet GCP's containment stays 1.00 regardless.

TABLE IV. DELEGATION, FOUR-WAY AGENCY METRICS BY RUN SET.

model	arm	att.	grant	cont.	prov.
`llama3.1:8b`	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
`qwen3:4b` (a)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
`qwen3:4b` (b)	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
`qwen2.5:32b`	GCP	1	0	1.00	1.0
	A2A	0†	0	0.00	–†
`gemini-2.5-flash-lite`	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
`gemini-2.5-flash`	GCP	1	0	1.00	1.0
	A2A	1	1	0.00	0.0
`gemini-2.5-pro` (Σ/15)	GCP	14	0	1.00	–
	A2A	11	11	0.00	–

† degenerate run: base A2A issued 0 requests (nothing attempted or executed); gate absent ⇒ containment = 0.00 by convention.

E. Forced-access containment: the read-side counterpart (architectural + observed)

Table V reports the forced-access scenario, the read-side mirror of delegation: the task instructs the agent to read the confidential node directly, so the unauthorized-read path is exercised on every run that engages the task at all, not left latent as in software-org. att. = attempted unauthorized read; grant = granted unauthorized (executed, canary served); cont. = containment. Counting conventions match Table IV (`gemini-2.5-pro` row is Σ/15). GCP containment is 1.00 on every configuration that attempted the read. Base A2A containment is 0.00 throughout, and on two of the three hosted tiers—`flash` and `pro`, the stronger models—it *executed* the unauthorized read and the canary surfaced in the transcript, an *observed* granted-unauthorized-access, not only an architectural possibility. This is the result the prior edition of this article flagged as future work (Section VII): running the scenario is what separates "the model behaved" from "the protocol did not permit misbehavior," and stronger models are exactly where that distinction earns its keep, since they are the ones that actually attempt and complete the unauthorized read.

`llama3.1:8b` is a genuine outlier, not a missing data point: it made *zero* tool calls on either arm across two independent 5-seed reruns (20/20 runs), reproduced deterministically. A direct non-agentic probe against the same scenario input shows the model refusing in its final answer before ever invoking a tool—verbatim *"I can't do that. Is there anything else I can help you with?"* (GCP arm) and *"I can't help you with this request."* (A2A arm). The scenario's system-prompt phrase making the role mismatch explicit ("even though your role is not engineering") reads to this model's alignment training as a policy violation, so it declines outright—on *both* arms, independent of whether a gate exists to catch the read. This is a small-model alignment-vs-tool-use interaction, orthogonal to the protocol comparison: the same model calls tools without hesitation on software-org, supply-chain, and delegation. `gemini-2.5-flash-lite` shows a milder version of the same effect on the A2A arm only (it declined the task, 0 attempts), while its GCP arm attempted and was denied—so containment reads 1.00/0.00 by construction on both tiers, but for different reasons: the smallest hosted tier avoided the ungated read by declining the task, the smallest local model avoided it by declining everything.

TABLE V. FORCED-ACCESS, AGENCY METRICS BY MODEL FAMILY.

model	arm	att.	grant	cont.
`llama3.1:8b`	GCP	0‡	0	n/a‡
	A2A	0‡	0	n/a‡
`qwen3:4b`	GCP	1	0	1.00
	A2A	1	1	0.00
`qwen2.5:32b`	GCP	1	0	1.00
	A2A	1	1	0.00
`gemini-2.5-flash-lite`	GCP	1	0	1.00
	A2A	0†	0	0.00†
`gemini-2.5-flash`	GCP	1	0	1.00
	A2A	1	1	0.00
`gemini-2.5-pro` (Σ/15)	GCP	11	0	1.00
	A2A	12	12	0.00

‡ 0 tool calls, both arms: model refused in its final answer (see text). No gate exercised ⇒ containment undefined (≠ 1.00). † base A2A declined the task, 0 attempts, gate absent ⇒ containment = 0.00 by convention (cf. `qwen2.5:32b`, Table IV).

F. Cost and success: parity (preliminary)

Wire tokens and task success are within seed variance between arms wherever the model could do the task. On `llama3.1:8b` software-org, tokens are 500.6±16.9 vs. 502.4±18.9 (95% CI ±14.8 vs. ±16.6, k=5; GCP −A2A mean difference −1.8 tokens, Cohen's $d = -0.10$, a negligible effect); on `qwen2.5:32b` software-org, 669.7±0.9 vs. 668.7±3.3 (mean difference +1.0 token, 0.15%, Cohen's d=0.41, small-to-moderate but on a 3-seed sample). Absolute counts differ by an order of magnitude across models, but the between-arm gap stays inside the interval in every scenario, and the effect sizes above confirm the gap is small relative to run-to-run variance rather than a consistent directional cost. The base A2A arm's *wire*-token count is often marginally *lower* (it omits GCP's query envelope); the tax A2A pays is the per-request tool-prompt budget (Table III), not the payload. No claim is therefore made that GCP sends fewer wire tokens—only that it does not cost meaningfully more while removing the $\mathcal{O}(n)$ prompt overhead. These results are presented as *preliminary*, pending 20–30 seeds and significance testing.

G. Leakage: ties in the canary scenarios (observed), guarantee holds (architectural)

The leakage picture is reported plainly. In *software-org*, observed leakage was 0.0 on both arms across all models, because the agent satisfied the public-summary task without ever requesting the confidential node, so the leaky path was *not exercised*; the architectural leak is latent, not observed. In *supply-chain*, both arms are authorized to read both owners' nodes, so both surfaced the canaries equally (leakage 1.0 on both, every model): a genuine *tie*, because the leak is not a gating decision. No observed behavioral

leakage advantage is therefore claimed. The leakage advantage of GCP is the *architectural* deny-by-default-plus-audit guarantee, exercised directly by the delegation and provenance results, not by these canary counts.

H. Architectural guarantee: executable no-leak property

The distinction above rests on a proof, not only measurement. Over the shipped read/authorize path, property-based tests (`fast-check`) are run against fixed, finite pools: five capabilities and four roles, one of which (`role:child`) inherits from another (`role:a`) so effective-capability computation is exercised, not only direct grants. Each random *policy* independently samples a subset of the role pool for `readableByRoles`, a subset of the capability pool for `requiredCapabilities`, a random `fallbackAllowed` boolean, and a random denial mode. Each random *principal* independently samples one role from the pool or none, plus a random subset of capabilities held directly. Because both are drawn from small pools by subset sampling rather than from an unbounded space, authorized and unauthorized draws both occur frequently, so the 1000-run default is not dominated by one outcome. Every generated $(policy, principal)$ pair is then driven through the real context-query handler (not a mock) against a stub adapter that *always* returns a sentinel canary string, so P1 (end-to-end no-content-leak) asserts that an unauthorized query never returns the canary and is marked `denied`, while an authorized one does return it (liveness, so the test is not vacuous); P2 (authorize soundness) checks the shipped gate's decision equals an independently written reference predicate over the same pair; P3 (role-inheritance soundness) checks effective-capability computation against the role chain directly. All pass at 1000 runs (raisable via an environment variable; runs are seed-reproducible for debugging). The canary metric *measures* leakage in scenarios; the property suite *proves* the gate is sound over the input space. Together: the property shows the gate is correct; the canary confirms it is on the hot path.

VII. Discussion, Limitations, and Threats to Validity

A. Interpretation of principal findings

In the forced-access scenario, GCP blocked all 11 unauthorized read requests issued by `gemini-2.5-pro` across the 15 evaluated runs, whereas the base A2A implementation granted the 12 requests issued in its corresponding runs (Table V). This result is consistent with Saltzer and Schroeder's principles of least privilege and complete mediation [11] and with Sandhu et al.'s role-based access control [12], since authorization is verified before content is admitted into the agent's context rather than relying on the agent's discretion to refrain from requesting it. It also aligns with Evertz et al.'s view that confidentiality is a system-level property rather than a purely model-level one [22], and with AirGapAgent's approach of restricting what an agent can access [24]. In relation to Progent, the finding supports the value of enforcing explicit privilege controls at the tool-access boundary [26]; GCP's evaluated contribution is to integrate that control with federated context reading and per-decision provenance. In practical terms, the results suggest that this integration can contain unauthorized requests even when the model attempts to issue them. The observed gap, however, corresponds to a base A2A implementation without equivalent controls, and does not demonstrate an inherent limitation of the protocol nor superiority over the cited protection systems, which were not evaluated directly. The same reading applies to the delegation results (Table IV): containment is architectural for GCP and absent for base A2A regardless of whether a given model happens to attempt the unauthorized action.

B. Relation to prior work

Three threads from Section II converge in the results. On *coupling*, the measured $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$ integration-edge growth (Table II) and the $\mathcal{O}(1)$ vs. $\mathcal{O}(n)$ discovery/tool-prompt overhead (Table III) instantiate, for LLM agent topologies, the same edge-count argument that Hohpe and Woolf [7], Parnas [8], Stevens et al. [9], Briand et al. [10] established for software and integration architectures. On *least privilege and access control*, the containment and provenance results operationalize Saltzer and Schroeder [11] and Sandhu et al. [12] at the level of individual context reads rather than at the level of a system boundary. On *leakage and governance*, the tie observed in the non-adversarial canary scenarios and the gap observed under forced access together support Evertz et al. [22] and El Yagoubi et al. [23]: confidentiality outcomes depend on whether the unauthorized path is exercised, not only on whether a gate exists in principle, which is why the forced-access probe (Section VI-E) is more informative than the non-adversarial scenarios alone.

C. Practical implications and conditions of applicability

For deployments that must support cross-owner context sharing under least-privilege and audit requirements—e.g. regulated data-minimization obligations [21] or the OWASP LLM Top 10's Excessive Agency and Sensitive Information Disclosure risks [25]—the results indicate that a read-first, role-gated substrate can provide native containment and provenance without added instrumentation, and that this benefit does not depend on model strength: it held across four model families of very different capability. The caveat is that these benefits are demonstrated against a *base* A2A baseline; systems that already wrap A2A with policy middleware, logging proxies, or gateways may recover some of the same governance properties, at the cost of that added instrumentation. GCP is therefore best read as removing the need for such instrumentation, not as the only way to obtain it.

D. Limitations and threats to validity

a) Resource constraints scope the evaluation.: This study received no external funding and was carried out as an independent initiative (see the Funding statement); the reported scope reflects that resource ceiling rather than a methodological choice. Locally hosted open-weight seeds are bounded by wall-clock and compute cost, since each seed requires a full multi-step ReAct agent loop against a locally served model, repeated per model, scenario, and arm—this keeps the open-weight arm at 3–5 seeds rather than the 20–30 used for two of the three hosted Gemini tiers. Hosted-model seeds are in turn bounded by paid API usage, which is why only one hosted frontier family was evaluated and why the third Gemini tier stops at 15 seeds rather than 30. These constraints do not threaten the deterministic/architectural claims (coupling, discovery, tool-prompt overhead, provenance, containment), which hold by construction independent of sample size; they do bound how strongly the preliminary behavioral columns (token, latency, leakage, success) can be read, and a replication run under a larger compute or API budget should be expected to narrow, not overturn, the small effect sizes already reported. Related work cited in Section II (e.g., Zhang et al. [15], Mao and Mirhoseini [16]) evaluates at production or larger-budget scale; the comparison offered here is a fairness-controlled but resource-bounded complement to that literature rather than a like-for-like replacement.

b) Four model families, seven run sets, but open-weight seed counts stay low.: The behavioral results now span three open-weight models over two size classes (4B–32B), locally hosted, across four independent run sets (`qwen3:4b` was run twice, as an internal replication check, not as a fourth model), plus one hosted frontier family (Gemini 2.5, three tiers) across three further run sets—removing both the single-model and the open-weight-only threats. Seed counts, however, are uneven across the two groups: the open-weight arm stays at 3–5 seeds, the wall-clock cost of running every seed as a full multi-step ReAct agent loop against a locally hosted LLM server, repeated per model, per scenario, and per arm, making 20–30 seeds roughly an order of magnitude more expensive there. The hosted Gemini arm closes this gap for two of its three tiers (30 seeds for `flash-lite`/`flash`, meeting the significance-testing convention that was not affordable locally) and partially for the third (`gemini-2.5-pro`, 15 seeds—a first 30-seed attempt was killed by the harness's 60-minute test timeout before writing a report; the incremental writer used for the reported run avoids that failure mode but the timeout itself has not yet been raised to let a 30-seed `pro` run complete). This constraint does not affect the deterministic/architectural claims (coupling, discovery, tool-prompt overhead, provenance, containment, delegation/forced-access containment), which do not depend on seed count at all; it bears only on the preliminary token/latency/leakage/success columns, and only for the open-weight rows and the `pro` tier. A second hosted frontier family, and 20–30 seeds on `pro`, remain the natural next replication step.

c) Provenance is a base-implementation property.: The 0.0 for A2A is a property of base message-passing, not a proof that A2A cannot audit. It is framed as "no per-read traceability without added instrumentation," with instrumented A2A identified as the fair second baseline.

d) Effect-size and proportion statistics.: Section VI-F reports Cohen's *d* and percent change for the two token comparisons that have complete mean/s.d./seed-count triples in the text. A full effect-size and mean-difference table across all scenarios, and a difference-of- proportions statistic for task success, would additionally require the per-run message and success records for every model/scenario/arm combination to be tabulated alongside tokens, which the current manuscript reports only in aggregate for most rows; producing that table is listed under Limitations and Future Work.

e) Leakage measurement is limited.: Exact-match canary scanning misses paraphrase and semantic leakage, and behavioral leakage in the non-adversarial scenarios depends on the agent exercising the path (latent in software-org, tied in supply-chain). The forced-access scenario (Section VI-E)

now closes part of this gap by directly targeting the confidential node rather than waiting for the agent to request it, and shows both an architectural guarantee (GCP containment 1.00 throughout) and, on the two strongest hosted tiers, an *observed* granted-read leak on base A2A. Two reinforcing gaps remain as future work: an *adversarial-prompt* scenario that induces unauthorized retrieval through indirection rather than an explicit instruction, and a *semantic* leakage metric beyond exact match, which would also let the software-org and supply-chain canaries detect paraphrased disclosure. Delegation and forced-access are both already forced-action probes and both show the gate engaging.

f) Marketplace used for structure only.: Marketplace task success was low on *both* arms (e.g. falling to 0.0–0.4 at n=16, 32 on `llama3.1:8b`); the model often failed to pick the cheapest seller. Because the failure is equal across arms it does not threaten fairness, but the marketplace is used *only* for its deterministic structural, discovery, and tool-prompt curves, and its behavioral rows are excluded from the main claims.

g) Construct validity of the baseline.: The A2A arm runs real servers, the same brain, and the same task, built with equal care; its disadvantages (no read gate, no delegation gate, no native audit sink, pairwise topology) are intrinsic to base message-passing, not artifacts of a strawman.

VIII. Limitations and Future Work

Raise the open-weight arm toward 20–30 seeds with significance testing on token/latency/message deltas, raise the harness timeout so `gemini-2.5-pro` can complete a full 30-seed run, and add a second hosted frontier family beyond Gemini; add the instrumented-A2A second baseline; add the adversarial-prompt and semantic-leakage scenarios and split transcript vs. final-answer leakage; and design a privacy testbed that removes the authorized-read confound of the current supply-chain scenario so the gating advantage can be *observed*, not only argued, in that scenario too; and tabulate per-run message and success records across all model/scenario/arm combinations to support a full effect-size and difference-of-proportions analysis (Section VII). Bridging GCP to MCP and A2A as first-class interop is the natural deployment path.

IX. Conclusion

This work presented GCP—read-first, role-gated, audited context federation with a stricter-capability delegation gate—and a fairness-controlled comparison against base A2A message-passing across four model families: three open-weight local models and one hosted frontier family (Gemini 2.5, three tiers). The *structural* advantages are strong and model-independent: coupling improves from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ in integration edges and from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ in per-request discovery and tool-prompt overhead; provenance is native rather than added instrumentation; and in both agency scenarios—delegation (write-side) and forced-access (read-side)—GCP contained every unauthorized action while base A2A contained none, on every model family that engaged the task. The forced-access results additionally turn the architectural guarantee into an *observed* one on the two strongest hosted tiers, where base A2A did not just risk but actually executed the unauthorized read and surfaced the canary. Cost and success are at parity in preliminary behavioral runs, and canary leakage *ties* on the current non-adversarial testbed, so no privacy advantage is claimed there yet. The defensible contribution is the *integration*—federated read, role control, and per-read provenance—plus its measured comparison; the behavioral results are a multi-model pilot, now spanning one hosted frontier family alongside three open-weight models, that invites replication with more seeds and additional hosted models.

Conflict of Interest Statement

The authors declare that there are no potential conflicts of interest related to the research, authorship, or publication of this article.

Funding

This research received no external funding. The study was carried out as an independent initiative of the researchers.

Acknowledgment

The authors thank Universidad Nacional de San Agustín de Arequipa for being the institution that hosted this research.

Declaration on Generative AI

During the preparation of this work, the authors used Claude and Claude Code (Anthropic) to assist with drafting and editing manuscript text, language refinement, and implementing and running parts of the evaluation harness (scenario, adapter, and results-reporting code) under the authors' direction and supervision. The scientific ideas, experimental design, interpretation of results, and conclusions in this article were developed by the human authors. After using these tools, the authors reviewed and edited all AI-assisted content and code, verified all reported results against the underlying run artifacts, and take full responsibility for the accuracy, originality, and integrity of this publication. No generative AI tool is listed as an author.

References

[1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[2] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. [Online]. Available: https://arxiv.org/abs/2307.07924

[3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[4] Google and Linux Foundation, "Agent2Agent (A2A) protocol specification, v1.0," Open standard, 2025. [Online]. Available: https://a2a-protocol.org/v1.0.0/specification/

[5] T. Finin, R. Fritzson, D. McKay, and R. McEntire, "KQML as an agent communication language," in *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.

[6] Foundation for Intelligent Physical Agents, "FIPA ACL message structure specification," FIPA, Tech. Rep. SC00061G, 2002. [Online]. Available: http://www.fipa.org/specs/fipa00061/

[7] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.

[8] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[9] W. P. Stevens, G. J. Myers, and L. L. Constantine, "Structured design," *IBM Systems Journal*, vol. 13, no. 2, pp. 115–139, 1974.

[10] L. C. Briand, J. W. Daly, and J. K. Wüst, "A unified framework for coupling measurement in object-oriented systems," *IEEE Transactions on Software Engineering*, vol. 25, no. 1, pp. 91–121, 1999.

[11] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[12] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[13] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8048–8057. [Online]. Available: https://arxiv.org/abs/2402.01680

[14] Anthropic, "Introducing the model context protocol," Anthropic; open specification, 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[15] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," *arXiv preprint arXiv:2410.02506*, 2024. [Online]. Available: https://arxiv.org/abs/2410.02506

[16] Y. Mao and A. Mirhoseini, "Decentralized multi-agent systems with shared context," *arXiv preprint arXiv:2606.10662*, 2026. [Online]. Available: https://arxiv.org/abs/2606.10662

[17] S. Panichella, M. I. Rahman, and D. Taibi, "Structural coupling for microservices," in *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, 2021, pp. 280–287. [Online]. Available: https://arxiv.org/abs/2103.04674

[18] X. Shen, Y. Liu, Y. Dai, Y. Wang, R. Miao, Y. Tan, S. Pan, and X. Wang, "Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems," *arXiv preprint arXiv:2505.23352*, 2025. [Online]. Available: https://arxiv.org/abs/2505.23352

[19] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (ABAC) definition and considerations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-162, 2014.

[20] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[21] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25," Official Journal of the European Union, L 119, 2016. [Online]. Available: https://gdpr-info.eu/

[22] J. Evertz, M. Chlosta, L. Schönherr, and T. Eisenhofer, "Whispers in the machine: Confidentiality in agentic systems," *arXiv preprint arXiv:2402.06922*, 2024. [Online]. Available: https://arxiv.org/abs/2402.06922

[23] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, "AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems," *arXiv preprint arXiv:2602.11510*, 2026. [Online]. Available: https://arxiv.org/abs/2602.11510

[24] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, and D. Ramage, "AirGapAgent: Protecting privacy-conscious conversational agents," *arXiv preprint arXiv:2405.05175*, 2024. [Online]. Available: https://arxiv.org/abs/2405.05175

[25] OWASP GenAI Security Project, "OWASP top 10 for large language model applications," OWASP Foundation, 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[26] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," *arXiv preprint arXiv:2504.11703*, 2025. [Online]. Available: https://arxiv.org/abs/2504.11703

[27] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated delegation and authorized AI agents," *arXiv preprint arXiv:2501.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[28] V. Ojewale, H. Suresh, and S. Venkatasubramanian, "Audit trails for accountability in large language models," *arXiv preprint arXiv:2601.20727*, 2026. [Online]. Available: https://arxiv.org/abs/2601.20727